

C Programming

Contents

Formatted I/O

Compatibility

C string and dynamic memory

File I/O

References

Programming in C – UNIX System Calls and Subroutines using C by A. D. Marshall

<http://www.cs.cf.ac.uk/Dave/C/CE.html>

The GNU C Library

http://www.gnu.org/software/libc/manual/html_node/index.html

The C Book

http://publications.gbdirect.co.uk/c_book/

Programming in C

Most operating systems are implemented in lower level languages
C and assembly are the common choices

You've acquired the basic programming skill in Python. In here, we are going to give you an overview of C programming.

Structure of C Programs

The program structures of C and C++ are almost identical

```
// Example1.c

#include <stdio.h>

int main()
{
    printf( "Beware the differences between C & C++ !!\n" );
    getchar();
    return 0;
}
```

To compile, use "gcc"

gcc *Example1.c -o example1*

To run it

./example1

I/O

For output to screen, use printf()

```
int printf(char *format, arg list ...)
```

This function takes a **variable number** of arguments

Prints to standard output the list of arguments according to the **specified format string**

The first one is a string describing the formatting information of the rest of the arguments

For more details, read its Linux manpage:

man 3 printf

printf()

In the formatting string, the character “%” is special

The character that follows it describes the format of a value to be displayed.

%d for int, %f for float, %s for string, %c for char

Example

```
int abc=10;  
float efg=6.54321;  
printf("integer is: %4d, float is: %.2f, string is: \" %s \" \n", abc,  
efg, "So what");
```

Output is:

```
integer is:  10, float is: 6.54, string is: " So what "
```

scanf()

For input from keyboard, use scanf() or . . .

man 3 scanf

`int scanf(char *format, args....)`

Reads characters from standard input, interprets them according to the specified format, and stores the results to variables specified in args list.

Format control string similar to printf()

Example

```
// Example2.c
#include <stdio.h>

int main()
{
    int number;
    float fnum;

    printf( "Please enter a number then follow by a float number: " );
    scanf( "%d %f", &number, &fnum );
    printf( "You entered %d & %f\n", number, fnum );
    getchar();
    return 0;
}
```

sprintf()

```
int sprintf(char *s, const char *format,
...);
```

Identical to printf except that the output is not written to the stdout, but written into the character array pointed to by s.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char mystr[50];
```

```
    int number;
```

```
    float fnum;
```

```
    printf( "Please enter a number then follow by a float number: " );
```

```
    scanf( "%d %f", &number, &fnum );
```

```
    printf( "You entered %d & %f\n", number, fnum );
```

```
    sprintf(mystr, "You entered %6d & %.4f\n", number, fnum);
```

```
    printf("%s", mystr);
```

```
    return 0;
```

```
}
```

Please enter a number then follow by a float number: 123 98.654321

You entered 123 & 98.654320

You entered 123 & 98.6543

Compatibility (1)

In earlier versions of C, declarations could appear only at a block's beginning, before any other statements.

```
//Example3.c

#include <stdio.h>
int main()
{
    /* This may be wrong!
       The variable declaration must appear first */
    printf( "Declare x next" );
    int x;

    return 0;
}
```

Compatibility (1)

It is allowed to mix declarations and code for the latest C standard (C99), and gcc has supported this feature

We recommend to be aware of the incompatibility of some C compilers, which not yet support the new feature

Keep the traditional approach would avoid those incompatibility problem

Also this keeps your code more manageable

Compatibility (2)

The loop-variable declaration in for-loop is not allowed in earlier versions of C.

```
//Example4.c

#include <stdio.h>
int main()
{
    /* this is not allowed in early version */
    for (int ind=0; ind<10; ind++)
        printf("%d\t", ind);
    printf("\n");

    return 0;
}
```

Now, gcc supports this way of declaring the variables.

```
gcc Example4.c -o ex4
```

char *

C does not have the data type string

In C, strings are really **arrays of chars** with the **last characters** end with '\0'

To manipulate the string, you can make use of library calls provided by the C library

#include <**string.h**>

string.h is a header file that contains many functions for manipulating strings

Functions such as

strcat, strcmp, strcpy, strlen, strtok, memcpy, memset, . . .

<https://users.cs.cf.ac.uk/Dave.Marshall/C/node19.html#SECTION00191000000000000000>

```
char *strtok(char *s1, const char *s2)
```

strtok()

Breaks the string pointed to by s1 into a sequence of tokens, each of which is delimited by one or more characters from the string pointed to by s2

Please enter a sentence: I am a happy boy

```
1 I
2 am
3 a
4 happy
5 boy
```

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    char mystr[50];
    char *piece;
    int i = 1;

    printf( "Please enter a sentence: " );
    fgets(mystr, 50, stdin);

    if (strlen(mystr) > 0) {
        piece = strtok(mystr, " ");
        while (piece) {
            printf("%d %s\n", i++, piece);
            piece = strtok(NULL, " ");
        }
    }

    return 0;
}
```

Dynamic Memory

In C, dynamic memories are allocated using the library call malloc() (or calloc())

```
void *malloc(size_t size);  
man malloc
```

To release the memory to the system, use free()

```
void free(void *ptr);
```

void * - a generic pointer

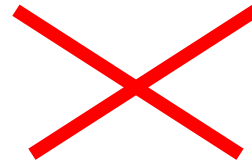
A pointer of type void * can have the value of any other pointer assigned to and can, conversely, be assigned to any other pointer

```
//create an array of 10 integers  
//notice the use of sizeof for portability  
int *data = (int*) malloc(sizeof(int) * 10);  
  
free(data);
```

Memory Allocation

One typical mistake is – the use of an array (actually a pointer) before creating memory space

```
int i;  
char * Aarray;  
for (i=0; i<length; i++)  
    Aarray[i] = '\\0';
```



Sometimes, the error does not pop up immediately; that would cause you lots of time to debug it

Always allocate memory storage before using the array and free the space after used

```
char * chararray;  
int * intarray;  
chararray = (char *) malloc(sizeof(char)*256);  
intarray = (int *) malloc(sizeof(int)*64);  
:  
free(chararray);  
free(intarray);
```

Parameter passing

C supports only pass-by-value parameter passing

To simulate pass-by-reference, pointers are used

```
void swap(int * x, int * y){  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}  
  
int a = 3, b = 5;  
swap(&a, &b);
```

File I/O

For C File I/O, we use a FILE object (stream) to keep track of the file being accessed

We can open a file, read the file, write to a file (depends on the rw status), and close the file after use

We have to pass the FILE ptr to all functions that do the actual operations

```
File *fopen(const char *filename, const char *mode)
int fclose(FILE *stream)
int fgetc(FILE *stream)
char *fgets(char *s, int count, FILE *stream)
int fputc(int c, FILE *stream)
int fputs(const char *s, FILE *stream)
int fprintf(FILE *stream, const char *format, arg list. . .)
int fscanf(FILE *stream, const char *format, arg list . . .)
```

File I/O

fopen - Modes of operation:

- r - open for reading
- w - open for writing (file need not exist)
- a - open for appending (file need not exist)
- r+ - open for reading and writing, start at beginning
- w+ - open for reading and writing (overwrite file)
- a+ - open for reading and writing (append if file exists)

fgetc - returns an integer ??

Reads the next character as an unsigned char from the File stream

fgets

Reads in characters from the File stream up to and including a newline character and stores them in the char array s, adding a null character to mark the end of the string

How long will the string be? At most count-1 in length

Standard Streams

There are three predefined streams open and available for use
FILE *stdin, FILE *stdout, and FILE *stderr

To output a char to standard output

```
int fputc (int c, FILE *stream)
```

```
int putchar (int c)
```

```
int fprintf(FILE * stream, const char * format, . . . );
```

We may generalize input/output using only file concept

Low-level I/O

File Streams provide a higher-level interface, layered on top of the primitive file descriptor facilities

Both file descriptors and file streams can represent a connection to a device / a pipe / a socket / a normal file

Stream-level I/O is more flexible and usually more convenient

Unless, if you want to

- do control operations that are specific to a particular kind of device, you must use a file descriptor

- read an entire file into memory before parsing it

- pass descriptors to a child process as it cannot inherit a stream directly

File Descriptor

Which is a unique integer number given to and for identifying each opened “file”

To open a file

```
int open (const char *filename, int flags[, mode_t mode])
```

The flags argument controls how the file is to be opened

The flag value is created by the bitwise OR of the appropriate parameters

`O_APPEND, O_CREAT, O_EXCL, O_RDONLY, O_RDWR, O_WRONLY, . .`

The mode argument is related to the access permission, and is only relevant if O_CREAT flag is specified

To close an opened file

```
int close (int fildes)
```

I/O Operations

read() gets a block of information from an opened file

The data are loaded directly into memory (which is the supplied buffer) as a sequence of bytes

```
ssize_t read (int fildes, void *buffer, size_t size)
```

write() writes a block of *size* bytes from a contiguous portion of memory (pointed by the supplied buffer) to an opened file

```
ssize_t write (int fildes, const void *buffer, size_t size)
```

Using man Command

man - an interface to the on-line reference manuals

To look for an on-line manual of a function, e.g. fork

man fork

The system might have more than one manual for the function you are looking for

The manpage returned by the system may not be the one you need

To search for the function, e.g. open

man -f open

A list of available manpages for open will be listed out

Pick the one you want to display, e.g. the open() library function

man 2 open

Using man Command

For more information about using man
man man

Command Line Arguments

In C it is possible to accept command line arguments

e.g. `ps aux`; `ls -l`

To use command line arguments in your program, you must first understand the full declaration of the main function

```
int main (int argc, char *argv[ ])
```

`argc` - the number of arguments passed into the program from the command line, **including the name** of the program

e.g. `argc = 2` for `"ls -l"`; `argc = 4` for `"ls -l -t -r"`

`argv` - contains the full list of all of the command line arguments

e.g. `argv[0] = "ls"`; `argv[1] = "-l"`; `argv[2] = "-t"`; `argv[3] = "-r"`

Command Line Arguments

```
#include <stdio.h>

int main(int argc, char **argv)
{
    int i = 1;

    printf("You have entered %d arguments\n", argc);
    printf("They are:\n");
    printf("CMD: %s\n", argv[0]);
    while (i < argc) {
        printf("arg%d: %s\n", i, argv[i]);
        i++;
    }

    return 0;
}
```

```
Cprog$ ./a.out 12 abc 43 def
You have entered 5 arguments
They are:
CMD: ./a.out
arg1: 12
arg2: abc
arg3: 43
arg4: def
```

Boolean

Traditionally, C uses the convention that zero and NULL pointer are false in conditions; while the other values represent true

Boolean data type is included in C99.

If you “#include <stdbool.h>”, you can use C++-like keywords “bool, true, false”

The <stdbool.h> header shall define the following macros:

bool

Expands to _Bool.

true

Expands to the integer constant 1.

false

Expands to the integer constant 0.